

Predicting Student Performance



Angeleanne Enriquez
20024291



Jack Bulas
20013569



Alice Zaytseva
20014236

A group of heroes, a dataset, and a curse

<https://www.kaggle.com/datasets/jimarahman/student-exam-performance-dataset-analysis/data>

Student Exam Performance Dataset Analysis

Marks secured by the students in various subjects



Data Card Code (6) Discussion (0) Suggestions (0)

About Dataset

This dataset show the student-related academic, lifestyle, and socio-enconmic factores that include exam performance. It include study hours,student attendance,sleep hours,motivation,tutoring sessions,parental involvement,school type and other envirmmental features.The mein target varibale is Exam_Score with represents the student final exam score.

The Dataset can be used for:

- 1→Exploratory Data Analysis (EDA)
- 2→Regression modeling (score prediction)
- 3→Feature importance analysis
- 4→Educational performance insights

Usability ⓘ

10.00

License

MIT

Expected update frequency

Never

Tags

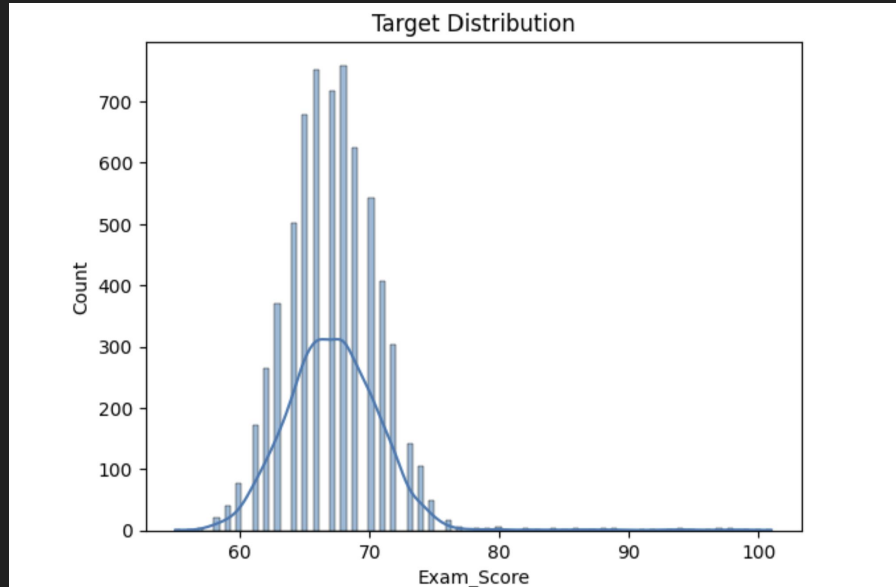
Tabular

Categorical

Multimodal

Objective

Given a set of characteristics relating to academic performance, use ML models to predict student outcomes (through grades).



Exam_Score
67
61
74
71
70
71
67
66
69

Features - All 19 of them

X.info()

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 6607 entries, 0 to 6606
```

```
Data columns (total 19 columns):
```

#	Column	Non-Null Count	Dtype
0	Hours_Studied	6607 non-null	int64
1	Attendance	6607 non-null	int64
2	Parental_Involvement	6607 non-null	object
3	Access_to_Resources	6607 non-null	object
4	Extracurricular_Activities	6607 non-null	object
5	Sleep_Hours	6607 non-null	int64
6	Previous_Scores	6607 non-null	int64
7	Motivation_Level	6607 non-null	object
8	Internet_Access	6607 non-null	object
9	Tutoring_Sessions	6607 non-null	int64
10	Family_Income	6607 non-null	object
11	Teacher_Quality	6529 non-null	object
12	School_Type	6607 non-null	object
13	Peer_Influence	6607 non-null	object
14	Physical_Activity	6607 non-null	int64
15	Learning_Disabilities	6607 non-null	object
16	Parental_Education_Level	6517 non-null	object
17	Distance_from_Home	6540 non-null	object
18	Gender	6607 non-null	object

```
dtypes: int64(6), object(13)
```

```
memory usage: 980.9+ KB
```

- Hours_Studied
- Attendance
- Parental_Involvement
- Access_to_Resources
- Extracurricular_Activities
- Sleep_Hours
- Previous_Scores
- Motivation_Level
- Internet_Access
- Tutoring_Sessions
- Family_Income
- Teacher_Quality
- School_Type
- Peer_Influence
- Physical_Activity
- Learning_Disabilities
- Parental_Education_Level
- Distance_from_Home
- Gender

First Drafts

Knn

MAE: 1.535451336359052
RMSE: 2.649983111108039
R2: 0.5135793236488211

Random Forest

MAE: 1.106490166414523
RMSE: 2.272102700316754
R2: 0.6424129925678828

Decision Tree

MAE: 1.7304248372827837
RMSE: 2.7439133386161134
R2: 0.4784852491184648

ANN

MSE: 4.390459060668945
MAE: 0.7696608304977417
RMSE: 2.095342229963627
62/62  0s 1ms/step
R2 Score: 0.695886492729187

Feature Analysis

- Our first step was classic feature analysis.
 - 19 features - surely there are some that are just noise.
- PCA
- Correlation between numerical features
 - Generate heatmap
- Conditional Entropy
 - Categorical features

PCA

Performed only on numeric features

6 principle components

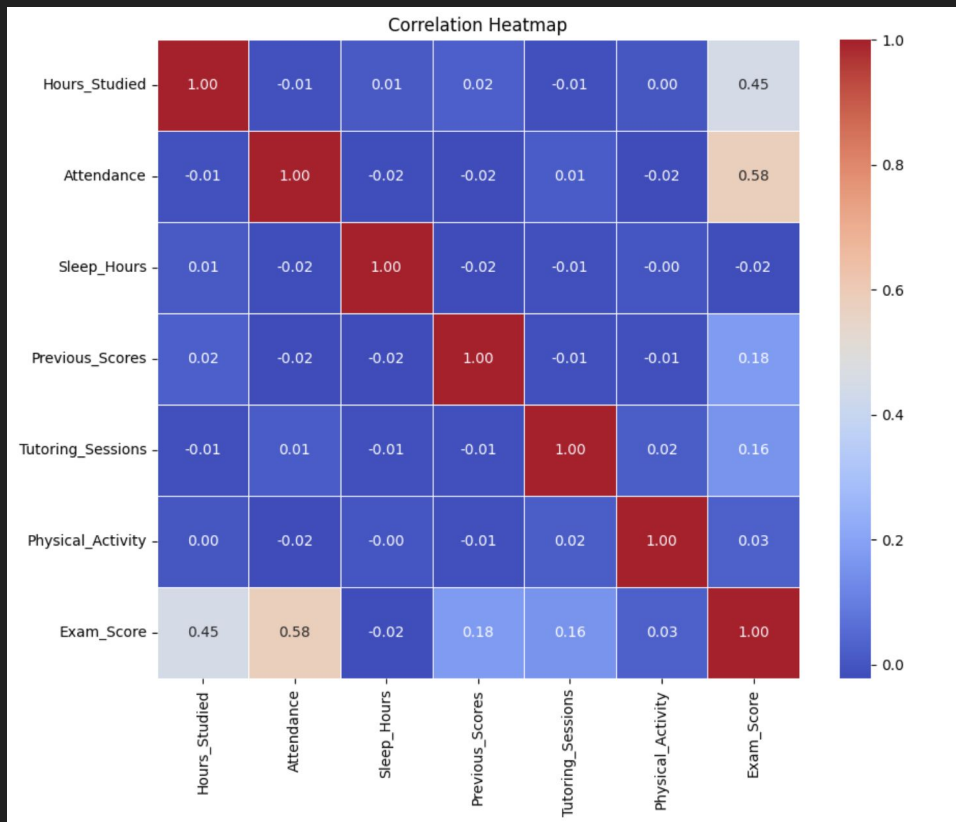
Each of relative significance

Each feature is significant
to multiple significant
components

Each numeric feature
seems important.

Explained variance ratio: [0.17499229 0.17117895 0.16953163 0.16470465 0.16174245 0.15785003] [0.17499229 0.34617124 0.51570287 0.68040752 0.84214997 1.]							
[38]:	PC1	PC2	PC3	PC4	PC5	PC6	
Hours_Studied	0.515308	0.005894	-0.043992	0.781560	-0.118354	-0.328092	
Attendance	-0.484795	-0.419780	0.201706	0.492801	-0.277194	0.477897	
Sleep_Hours	0.160264	0.507694	0.595897	0.161123	0.397419	0.421387	
Previous_Scores	0.507840	-0.414661	-0.378569	-0.067120	0.320077	0.565585	
Tutoring_Sessions	-0.464343	0.146510	-0.451198	0.332681	0.660409	-0.111915	
Physical_Activity	-0.015230	0.610400	-0.505364	0.071899	-0.461113	0.392418	

Numerical Feature Correlation



$r = +1$ — perfect positive relationship (as X increases, Y increases proportionally)

$r = -1$ — perfect negative relationship (as X increases, Y decreases)

$r = 0$ — no linear relationship

$r > 0.7$ — strong positive

0.4–0.7 — moderate

< 0.4 — weak

Categorical Feature Conditional Entropy

H(Y) marginal entropy: 1.9787 bits

	H(Y X)	Information_Gain	IG_Ratio
Access_to_Resources	1.954731	0.023950	0.012104
Parental_Involvement	1.960795	0.017886	0.009039
Peer_Influence	1.969415	0.009266	0.004683
Family_Income	1.971927	0.006754	0.003413
Motivation_Level	1.971965	0.006716	0.003394
Learning_Disabilities	1.973044	0.005638	0.002849
Extracurricular_Activities	1.975536	0.003146	0.001590
Internet_Access	1.976212	0.002469	0.001248
School_Type	1.978537	0.000144	0.000073
Gender	1.978610	0.000072	0.000036
Teacher_Quality	NaN	NaN	NaN
Parental_Education_Level	NaN	NaN	NaN
Distance_from_Home	NaN	NaN	NaN

High IG (e.g. > 0.2)

Keep feature — it strongly reduces score uncertainty

Low IG (e.g. < 0.05)

Consider dropping — it's mostly noise for your models

IG ≈ 0 The feature and exam score are nearly independent

Permutation Importance

```
from sklearn.inspection import permutation_importance

result = permutation_importance(knn_model, X_test, y_test, n_repeats=10)

for i in result.importances_mean.argsort()[::-1]:
    print(f"{X.columns[i]}: "
          f"{result.importances_mean[i]:.4f} +/- {result.importances_std[i]:.4f}")

Attendance: 0.4936 +/- 0.0236
Hours_Studied: 0.2476 +/- 0.0064
Previous_Scores: 0.0372 +/- 0.0066
Tutoring_Sessions: 0.0257 +/- 0.0086
Access_to_Resources: 0.0232 +/- 0.0063
Parental_Involvement: 0.0218 +/- 0.0071
Family_Income: 0.0127 +/- 0.0089
Extracurricular_Activities: 0.0089 +/- 0.0128
Learning_Disabilities: 0.0081 +/- 0.0032
Peer_Influence: 0.0045 +/- 0.0064
Motivation_Level: 0.0028 +/- 0.0075
Internet_Access: 0.0023 +/- 0.0026
Physical_Activity: 0.0005 +/- 0.0074
School_Type: -0.0002 +/- 0.0036
Sleep_Hours: -0.0029 +/- 0.0103
Parental_Education_Level: -0.0043 +/- 0.0059
Teacher_Quality: -0.0053 +/- 0.0054
Distance_from_Home: -0.0086 +/- 0.0062
Gender: -0.0116 +/- 0.0065
```

Also measures how much each feature contributes to predictability *after* training and testing a model

Randomly shuffles each feature's values across all samples and re-tests the model with the jumbled dataset

Importance mean: how much accuracy dropped after shuffling the values

- **Higher = feature contributes heavily to predictability**
- **Near-zero = feature does not contribute at all**
- **Negative = feature is causing noise and should be treated as zero**

Features

- Hours_Studied
- Attendance
- Parental_Involvement
- Access_to_Resources
- Extracurricular_Activities
- Sleep_Hours
- Previous_Scores
- Motivation_Level
- Internet_Access
- Tutoring_Sessions
- Family_Income
- Teacher_Quality
- School_Type
- Peer_Influence
- Physical_Activity
- Learning_Disabilities
- Parental_Education_Level
- Distance_from_Home
- Gender

Improved KNN Model Performance - Removed Features

MAE: 1.535451336359052

RMSE: 2.649983111108039

R2: 0.5135793236488211

MAE: 0.6859304084720127

RMSE: 0.8793593230080465

R2: 0.9464377811317346

Improved KNN Model Performance - Grid Search

```
[17] ✓ 31s from sklearn.model_selection import GridSearchCV

k_values = range(1, 21) # test k neighbors from 1 to 20
distance = ['manhattan', 'euclidean', 'minkowski']
weight = ['uniform', 'distance']

param_grid = {
    'knn__n_neighbors': k_values, # note: stepname__parameter
    'knn__metric': distance,
    'knn__weights': weight
}

# gridsearch the pipeline
clf = GridSearchCV(knn_model, param_grid, cv=5, scoring='neg_mean_squared_error')
clf.fit(X_train, y_train)

print(clf.best_estimator_.named_steps['knn'].n_neighbors) # best k
```

▼ ... 9

```
[19] ✓ 0s print(clf.best_estimator_.named_steps['knn'].metric)
print(clf.best_estimator_.named_steps['knn'].weights)
```

▼ euclidean
distance

MAE: 0.6513102301529451

RMSE: 0.8372445433909317

R2: 0.9514453903918719

Improved DT Model Performance - Features, Grid Search

MAE: 1.7304248372827837

RMSE: 2.7439133386161134

R2: 0.4784852491184648

MAE: 0.38682041565207814

RMSE: 0.8782725662304932

R2: 0.9465700891291016

Improved RF Model Performance - Features, Grid Search

MAE: 1.106490166414523

RMSE: 2.272102700316754

R2: 0.6424129925678828

MAE: 0.0024886535552193405

RMSE: 0.038900191667263787

R2: 0.9998951835860093

Improved ANN Model Performance - Features, Grid Search

```
def build_model(neurons_1=64, dropout_rate=0.1, learning_rate=0.001):  
    model = Sequential()  
    model.add(Input(shape=(X_train_tf.shape[1],)))  
    model.add(Dense(neurons_1, activation='relu'))  
    model.add(Dropout(dropout_rate))  
    model.add(Dense(32, activation='relu'))  
    model.add(Dropout(dropout_rate))  
    model.add(Dense(1, activation='linear'))  
    model.compile(optimizer=Adam(learning_rate=learning_rate), loss='mse')  
    return model
```


Improved ANN Model Performance - Features, Grid Search

```
from sklearn.model_selection import KFold
import numpy as np
from itertools import product

param_grid = {
    'neurons_1': [64, 128],
    'dropout_rate': [0.1, 0.2],
    'learning_rate': [0.001, 0.01]
}

kf = KFold(n_splits=5, shuffle=True, random_state=myFavoriteNumber)
results = []

keys, values = zip(*param_grid.items())
for combo in product(*values):
    params = dict(zip(keys, combo))
    fold_scores = []

    for train_idx, val_idx in kf.split(X_train_tf):
        X_fold_train, X_fold_val = X_train_tf[train_idx], X_train_tf[val_idx]
        y_fold_train, y_fold_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

        model = build_model(**params)
        model.fit(X_fold_train, y_fold_train, epochs=20, batch_size=8, verbose=0)

        mse = model.evaluate(X_fold_val, y_fold_val, verbose=0)
        fold_scores.append(np.sqrt(mse)) # RMSE

    mean_rmse = np.mean(fold_scores)
    results.append(**params, 'rmse': mean_rmse)
print(f"{params} → CV RMSE: {mean_rmse:.4f}")
```

Improved ANN Model Performance - Features, Grid Search

MSE: 4.390459060668945

MAE: 0.7696608304977417

RMSE: 2.095342229963627

62/62  0s 1ms/step

R2 Score: 0.695886492729187

MSE: 0.5684359669685364

MAE: 0.5787816643714905

RMSE: 0.7539469258300191

62/62  0s 2ms/step

R2 Score: 0.9606261849403381

Overall Best Model Performance: Random Forest Model

MAE: 0.0024886535552193405

RMSE: 0.038900191667263787

R2: 0.9998951835860093

Near perfect prediction rate (~99% R^2 score)

Predictions are only wrong ~.2% of the time (~0.002 MAE score)

No huge outlier errors (~.039 RMSE score)

Conclusion and Key Takeaways

Curse of Dimensionality: a dataset having a larger number of features leads to data points become equidistant/less similar to each other

A model's performance is only as good as its data → The importance of preprocessing data.

Other Takeaways

- Gender is irrelevant to grade performance
- No tree depth will save you
- Attend class, it *will* save you